King's College London
Department of Informatics
Dr. Mohammad Abdulaziz

Computer-Assisted Theorem Proving: 7CCSACTP

Exercise Sheet 7

Important Note: Please bring a laptop with Isabelle/HOL installed to the Large Group Tutorial (LGT). Performing these proofs in Isabelle will be an integral part of the tutorial.

In this sheet we use the 2-3 trees from the lecture: their height, the structural invariant *complete*, and their in-order traversal.

```
datatype 'a tree23 = Leaf
  | Node2 'a tree23 'a 'a tree23
  | Node3 'a tree23 'a 'a tree23 'a 'a tree23

fun height :: 'a tree23 ⇒ nat where
  height Leaf = 0
| height (Node2 l _ r) = max (height l) (height r) + 1
| height (Node3 l _ m _ r) = max (height l) (max (height m) (height r)) + 1
```

HOL already has a constant *complete* (for complete metric spaces); *hide_const* hides it, so that we can use the name for 2-3 trees.

```
hide_const (open) Topological_Spaces.complete

fun complete :: 'a tree23 ⇒ bool where
  complete Leaf = True
| complete (Node2 l _ r) = (height l = height r ∧ complete l ∧ complete r)
| complete (Node3 l _ m _ r) =
    (height l = height m ∧ height m = height r ∧ complete l ∧ complete m ∧ complete r)

fun inorder :: 'a tree23 ⇒ 'a list where
  inorder Leaf = []
| inorder (Node2 l a r) = inorder l @ a # inorder r
| inorder (Node3 l a m b r) = inorder l @ a # inorder m @ b # inorder r
```

Insertion, as in the lecture. HOL already has a function *insert*, which inserts an element into a set; *hide_const* hides it, so that we can use the name for 2-3 trees.

```
datatype 'a up_i = Eq_i 'a tree23 | Of 'a tree23 'a 'a tree23

fun tree_i :: 'a up_i ⇒ 'a tree23 where
  tree_i (Eq_i t) = t
| tree_i (Of l a r) = Node2 l a r
```

```
fun ins :: 'a::linorder ⇒ 'a tree23 ⇒ 'a up_i where
  ins x Leaf = Of Leaf x Leaf
| ins x (Node2 l a r) =
    (if x < a then (case ins x l of
                      Eq_i l' ⇒ Eq_i (Node2 l' a r)
                    | Of l1 b l2 ⇒ Eq_i (Node3 l1 b l2 a r)) else
     if x > a then (case ins x r of
                      Eq_i r' ⇒ Eq_i (Node2 l a r')
                    | Of r1 b r2 ⇒ Eq_i (Node3 l a r1 b r2))
     else Eq_i (Node2 l a r))
| ins x (Node3 l a m b r) =
    (if x < a then (case ins x l of
                      Eq_i l' ⇒ Eq_i (Node3 l' a m b r)
                    | Of l1 c l2 ⇒ Of (Node2 l1 c l2) a (Node2 m b r)) else
     if x = a then Eq_i (Node3 l a m b r) else
     if x < b then (case ins x m of
                      Eq_i m' ⇒ Eq_i (Node3 l a m' b r)
                    | Of m1 c m2 ⇒ Of (Node2 l a m1) c (Node2 m2 b r)) else
     if x = b then Eq_i (Node3 l a m b r)
     else (case ins x r of
             Eq_i r' ⇒ Eq_i (Node3 l a m b r')
           | Of r1 c r2 ⇒ Of (Node2 l a m) b (Node2 r1 c r2)))

hide_const insert

definition insert :: 'a::linorder ⇒ 'a tree23 ⇒ 'a tree23 where
  insert x t = tree_i (ins x t)
```

Exercise 7.1 Insertion Preserves Completeness

We use the following function, which returns the height of a result of *ins*:

```
fun h_i :: 'a up_i ⇒ nat where
  h_i (Eq_i t) = height t
| h_i (Of l a r) = height l
```

Prove that insertion preserves completeness, first pen-and-paper, then in Isabelle.

```
lemma complete_insert: complete t ⟹ complete (insert a t)
  sorry
```

Hints: 1. Induction on t directly for this statement gets stuck: in the step, you need to know that inserting into a subtree did not change its height. As in the lecture, first prove a stronger lemma about *ins*, which also says what happens to the height (use h_i).

2. *auto split*: up_i*.splits* lets *auto* split the *case* expressions on the result of *ins*. With it, the proof from the lecture takes about a minute. *auto split*!: *if_split* up_i*.split*, which breaks up the goal after every split of an *if* or *case*, takes a few seconds.

Exercise 7.2 The Height of 2-3 Trees

The height of a complete 2-3 tree is logarithmic in its size:

$$\log_3(|t| + 1) \le h(t) \le \log_2(|t| + 1)$$

In Isabelle, $|t|$ is *size t*, the number of nodes, which the *datatype* command defines automatically.

(a) Prove, first pen-and-paper and then in Isabelle:

```
lemma size_lb: complete t ⟹ 2 ^ height t ≤ size t + 1
  sorry

lemma size_ub: complete t ⟹ size t + 1 ≤ 3 ^ height t
  sorry
```

(b) Derive the logarithmic bounds:

```
lemma height_ub: complete t ⟹ height t ≤ log 2 (size t + 1)
  sorry

lemma height_lb: complete t ⟹ log 3 (size t + 1) ≤ height t
  sorry
```

Hints: 1. *size_lb* is the lemma from the lecture.

2. For (b), use (a) and sledgehammer. *find_theorems log _ ^ _* lists library lemmas relating logarithms and powers. *log* works on real numbers: Isabelle converts *size t + 1* to a real number automatically, written *real* (*size t + 1*). To turn an inequality between natural numbers into one between real numbers, use *of_nat_mono*.

Exercise 7.3 Joining 2-3 Trees

Define a function *join* that combines two complete 2-3 trees of the same height into one tree. The in-order traversal of the result must be the in-order traversal of the first tree followed by that of the second, and the height of the result must be either the height of the arguments or one more. As for *ins*, use *'a up$_i$* to return the result: *Of* signals that the height has increased.

```
fun join :: 'a tree23 ⇒ 'a tree23 ⇒ 'a up_i where
```

Prove that your function is correct:

```
lemma join_inorder:
  assumes height t1 = height t2 complete t1 complete t2
  shows inorder (tree_i (join t1 t2)) = inorder t1 @ inorder t2
```

sorry

lemma *join_complete*:
assumes *height t1 = height t2 complete t1 complete t2*
shows *complete* (*tree*$_i$ (*join t1 t2*)) $\wedge$ h_i (*join t1 t2*) = *height t2*
sorry

Hints: 1. Combine the two trees in the middle: recursively join the rightmost subtree of the first tree with the leftmost subtree of the second, then rebuild the outer nodes, splitting them if the result overflows.

2. Prove both lemmas by computation induction (*join.induct*), with *auto split*: *up*$_i$*.splits*. There are many cases: if *auto* fails, use an Isar proof to look at the cases one by one. This also helps to find mistakes in your definition of *join*.